

Uma Formulação Linear e um Algoritmo Exato para o Problema da Seleção de Pedidos Ótima

André Luiz Feijó dos Santos

Universidade Federal de Viçosa

Av. Peter Henry Rolfs, s/n, Campus Universitário 36570-900 - Viçosa - MG

andre.santos1@ufv.br

Pedro Fiorio Baldotto

Universidade Federal de Viçosa

Av. Peter Henry Rolfs, s/n, Campus Universitário 36570-900 - Viçosa - MG

pedro.baldotto@ufv.br

RESUMO

Neste trabalho, é proposto um *pipeline* de resolução para o Problema de Seleção de Pedidos Ótima (SPO). Este *pipeline* é formado por uma etapa de redução iterativa do espaço de busca, por meio de duas rotinas paralelas que resolvem subproblemas do SPO com um número fixado de corredores, seguida de uma reformulação linear para a modelagem do problema, podendo ser resolvida de forma eficiente em *solvers* modernos. Os resultados mostraram que esta abordagem exata é capaz de resolver instâncias consideravelmente grandes até a otimalidade em tempo hábil, encontrando soluções ótimas em 27 das 35 instâncias públicas.

PALAVRAS CHAVE. SPO, Agrupamento de Pedidos, Reformulação, Logística, Otimização.

Tópicos OD - Otimização Discreta, ONL - Otimização Não-Linear, POI - PO na Indústria

ABSTRACT

In this work, we propose a solution pipeline for the Optimal Order Selection Problem (OOS). This pipeline consists of an iterative reduction step of the search space, through two parallel routines that solve subproblems of the OOS with a fixed number of aisles, followed by a linear reformulation for modeling the problem, which can be efficiently solved using modern solvers. The results showed that this exact approach is capable of solving considerably large instances to optimality in a timely manner, finding optimal solutions in 27 out of 35 public instances.

KEYWORDS. OOSP, Order Batching, Reformulation, Logistics, Optimization.

Paper topics OD - Discrete Optimization, ONL - Non-linear Optimization, POI – OR in Industry

1. Introdução

Estratégias de seleção de pedidos para entrega são fundamentais no setor de comércio e transporte de encomendas, e têm sido amplamente estudadas ao longo de décadas. A adoção de uma técnica de coleta, juntamente à métrica para avaliar sua produtividade, está totalmente associada ao modelo de negócio de cada empresa, o que implica na formulação de diversos problemas computacionais para atender a essas demandas, como Problemas de Agrupamento de Pedidos [Pardo et al., 2024] e de Roteamento em Armazéns [Masae et al., 2020]. Este trabalho foca em resolver o Problema da Seleção de Pedidos Ótima (SPO).

O SPO foi proposto pelo MERCADO LIVRE¹, para o desafio de otimização do LVII Simpósio Brasileiro de Pesquisa Operacional (SBPO)², em janeiro de 2025. A seguir, o problema é descrito conforme apresentado originalmente.

Após a confirmação da compra pelo cliente, seu pedido é registrado em um *backlog* de preparação, para ser montado com base no prazo de entrega acordado. A etapa de processamento (montagem) de um pedido envolve localizar e coletar os itens que o compõem, para que, após preparado, possa ser enviado ao cliente.

Os itens disponíveis para montagem de pedidos são organizados em múltiplos corredores. Cada corredor contém um ou mais tipos de itens, e cada tipo de item pode ter exemplares distribuídos em diferentes corredores. Para que a etapa de montagem se torne mais eficiente, a coleta é realizada após selecionar um subconjunto dos pedidos no *backlog* (conjunto de pedidos pendentes) para serem preparados simultaneamente. A este subconjunto de pedidos, é dado o nome de *wave*. Para que uma *wave* seja válida, é preciso respeitar um limite mínimo e máximo de itens.

O objetivo do SPO é encontrar uma *wave* capaz de maximizar a produtividade da etapa de coleta, seguindo uma determinada métrica. Para esta aplicação, a métrica foi definida como a razão da quantidade de itens coletados pelo número de corredores visitados. Todos os itens coletados devem ser utilizados para montar pedidos e cada pedido é montado apenas uma vez.

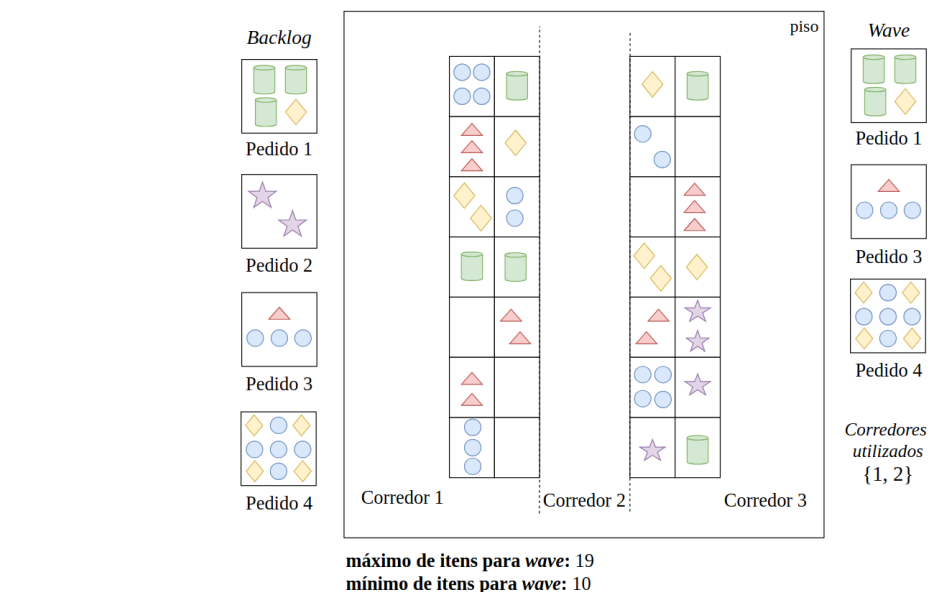
A Figura 1 exemplifica esse processo. São apresentados um *backlog*, contendo quatro pedidos em espera – cada um com seus respectivos itens necessários – uma possível distribuição de itens em corredores e, finalmente, uma possível *wave* formada pelos pedidos do *backlog*. O exemplo restringe o espaço de solução às *waves* com pelo menos 10 itens e, no máximo, 19. A *wave* escolhida é formada pelos pedidos 1, 3 e 4. Note que não seria possível montá-la visitando apenas um corredor, pois, para cada um, há pelo menos um pedido na *wave* que possui pelo menos um item cuja quantidade necessária é maior que a quantidade do mesmo no corredor. Porém, ao visitar apenas os corredores 1 e 2, é possível preparar a *wave*. Além disso, é possível montar exatamente o mesmo subconjunto de pedidos visitando, obrigatoriamente, todos os corredores, porém isso claramente reduz a produtividade.

Assim, o valor objetivo da solução apresentada pela Figura 1 é $\frac{17}{2} = 8.5$. É possível provar que essa solução é, de fato, ótima. Note, ainda, que a *wave* formada apenas pelo pedido 4, montado usando apenas itens do corredor 2, possui valor objetivo $\frac{9}{1} = 9$, porém é inviável por não respeitar o limite inferior de itens.

Neste trabalho, o SPO é formulado inicialmente como um problema de Programação Linear Inteira Mista Fracional (sigla em inglês, MILFP), uma classe de problemas $\mathcal{NP}$ -Difíceis, não convexos, com variáveis inteiras e contínuas, sujeito a restrições lineares e com objetivo definido pela razão entre duas funções [Yue et al., 2013]. Em seguida, é proposta uma reformulação para o problema, substituindo o objetivo fracional por uma função linear. Esse processo permite que o modelo seja resolvido de forma eficiente ao escrevê-lo como um problema de Programação Linear

¹<https://mercadolibre.com/>

²<https://sbpo2025.galoa.com.br/sbpo-2025/page/5407-home>



O SPO combina aspectos de *order batching* e *order picking*, porém, o faz de maneira singular, ao não requerer o roteamento da coleta de produtos, mas sim a minimização do número de corredores necessários, além de solicitar a formação de uma única *wave*, em uma formulação MILFP. Até onde é de nosso conhecimento, tais características conferem um caráter inédito à formulação do SPO, distinguindo-o dos problemas comumente abordados na literatura.

3. Formulação do Problema

Seja $\mathcal{A}$ o conjunto de corredores disponíveis, $\mathcal{I}$, o conjunto de todos os itens distribuídos por todos os corredores e $\mathcal{O}$, o conjunto de pedidos do *backlog*. Uma solução π para o SPO é uma tupla $(\mathcal{O}', \mathcal{A}')$, em que $\mathcal{O}' \subseteq \mathcal{O}$ e $\mathcal{A}' \subseteq \mathcal{A}$.

Considere, ainda, que $\mathcal{I}_o \subseteq \mathcal{I}$ é o conjunto de todos os itens solicitados para que o pedido o seja montado e $\mathcal{I}' = \bigcup_{o \in \mathcal{O}'} \mathcal{I}_o$, o conjunto de todos os itens coletados pela *wave*. Seja $w_{i,o}$ a quantidade do item i solicitado para o pedido o (se o não solicita i , $w_{i,o} = 0$) e $q_{i,a}$, a quantidade do item i disponível no corredor a (se a não oferta i , $q_{i,a} = 0$). Por fim, assuma $|\pi| = \sum_{o \in \mathcal{O}'} \sum_{i \in \mathcal{I}_o} w_{i,o}$ como o número de itens coletados para a *wave*. A título de abreviação, $|\pi|$ será chamada de “tamanho da *wave*”.

O objetivo do SPO é encontrar uma solução π que satisfaça

$$\max \frac{|\pi|}{|\mathcal{A}'|} \quad (1)$$

$$s.t. \quad \sum_{o \in \mathcal{O}'} w_{i,o} \leq \sum_{a \in \mathcal{A}'} q_{i,a}, \quad \forall i \in \mathcal{I}' \quad (2)$$

$$LB \leq |\pi| \leq UB \quad (3)$$

para $LB, UB \in \mathbb{Z}^+$, $LB \leq UB$, os limites inferior e superior, respectivamente, para o tamanho da *wave*. É dito que é possível montar os pedidos de $\mathcal{O}'$ utilizando os corredores em $\mathcal{A}'$ se, e somente se, $\mathcal{O}'$ e $\mathcal{A}'$ satisfizerem (2).

4. Modelo de Programação Linear Inteira Mista Fracional

Primeiramente, é proposto um modelo MILFP para resolver o SPO. Seja $y_a \in \{0, 1\}$ uma variável de decisão tal que $y_a = 1$ se o corredor a é utilizado na solução e 0, caso contrário, e p_o uma variável de decisão tal que $p_o = 1$ se o pedido o é montado na solução e 0, caso contrário. O seguinte modelo MILFP é formulado:

$$\max \frac{\sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times p_o}{\sum_{a \in \mathcal{A}} y_a} \quad (4)$$

$$s.t. \quad \sum_{o \in \mathcal{O}} w_{i,o} \times p_o \leq \sum_{a \in \mathcal{A}} q_{i,a} \times y_a, \quad \forall i \in \mathcal{I} \quad (5)$$

$$LB \leq \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times p_o \leq UB \quad (6)$$

$$y_a, p_o \in \{0, 1\}, \quad \forall o \in \mathcal{O}, a \in \mathcal{A} \quad (7)$$

A função objetivo (4) maximiza o valor da métrica de produtividade. A restrição (5) garante que o conjunto de pedidos da *wave* pode ser montado utilizando o conjunto de corredores. A restrição (6) garante que o tamanho da *wave*-solução respeita os limites impostos. A restrição (7) define o escopo das variáveis p_o e y_a .

Devido à natureza não convexa do modelo MILFP, podem ser utilizadas técnicas para transformar o programa em questão em um problema MILP. A seguir, é proposta uma formulação linear para o problema a partir do método da Reformulação-Linearização [Yue et al., 2013].

5. Método da Reformulação-Linearização

A Transformação de Charnes-Cooper [Charnes e Cooper, 1962] é um método de reformulação de problemas de Programação Linear Fracional em programas lineares. Quando variáveis inteiras são introduzidas ao modelo, este método não é mais suficiente. Neste caso, Yue et al. [2013] apresentam uma abordagem capaz de remover a divisão entre variáveis na função objetivo. Como o SPO contém apenas variáveis binárias, a aplicação deste método se torna simples e direta. A seguir, essa técnica é descrita.

São introduzidas ao modelo as variáveis contínuas $u = \frac{1}{\sum_{a \in \mathcal{A}} y_a}$, $t \in \mathbb{R}^{|\mathcal{O}|}$ e $g \in \mathbb{R}^{|\mathcal{A}|}$, tais que $t = u \times p$ e $g = u \times y$. Assim, a partir da função objetivo fracional (4), obtém-se a seguinte função linear:

$$\frac{\sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times p_o}{\sum_{a \in \mathcal{A}} y_a} \equiv u \times \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times p_o \equiv \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times t_o \quad (8)$$

Ainda, da definição de u , é possível concluir

$$u = \frac{1}{\sum_{a \in \mathcal{A}} y_a} \equiv \sum_{a \in \mathcal{A}} u \times y_a = 1 \equiv \sum_{a \in \mathcal{A}} g_a = 1 \quad (9)$$

que também é linear. Portanto, o programa MILFP pode ser linearizado conforme o modelo abaixo, ao qual nos referiremos como `ref-lin`.

$$\max \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times t_o \quad (10)$$

$$s.t. \quad \sum_{o \in \mathcal{O}} w_{i,o} \times t_o \leq \sum_{a \in \mathcal{A}} q_{i,a} \times g_a, \quad \forall i \in \mathcal{I} \quad (11)$$

$$LB \times u \leq \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times t_o \leq UB \times u \quad (12)$$

$$\sum_{a \in \mathcal{A}} g_a = 1 \quad (13)$$

$$t_o \leq p_o \quad \forall o \in \mathcal{O} \quad (14)$$

$$t_o \leq u \quad \forall o \in \mathcal{O} \quad (15)$$

$$t_o \geq u - (1 - p_o) \quad \forall o \in \mathcal{O} \quad (16)$$

$$g_a \leq y_a \quad \forall a \in \mathcal{A} \quad (17)$$

$$g_a \leq u \quad \forall a \in \mathcal{A} \quad (18)$$

$$g_a \geq u - (1 - y_a) \quad \forall a \in \mathcal{A} \quad (19)$$

$$y_a, p_o \in \{0, 1\}, \quad \forall o \in \mathcal{O}, a \in \mathcal{A} \quad (20)$$

$$g_a, t_o \geq 0, \quad \forall o \in \mathcal{O}, a \in \mathcal{A} \quad (21)$$

Assim como o modelo MILFP inicial, a função objetivo (10) maximiza a razão da métrica de produtividade. As restrições (11) e (12) preservam as condições antes impostas por (5) e (6). A restrição (13) garante a corretude do valor de u , e as restrições (14) a (16) e (17) a (19) são linearizações de $t = u \times p$ e $g = u \times y$, respectivamente.

6. Algoritmo Iterativo Paralelo

A resolução de um problema SPO possui duas camadas de otimização. A primeira, escolher os corredores que farão parte da solução. A segunda, quais pedidos serão montados. Cada

subconjunto de $\mathcal{A}$ com exatamente m corredores possui $\frac{|\mathcal{A}|!}{m!(|\mathcal{A}|-m)!}$ soluções, o que pode levar a espaços de busca significativamente grandes. Esta seção visa apresentar uma estratégia que pode ser aplicada tanto como simplificação do espaço de busca que será fornecido ao *solver* MILP quanto uma abordagem de resolução completa.

Para o Algoritmo Iterativo Paralelo, o qual chamaremos de *par-it*, é fixado o denominador da fração expressa em (4) e, para cada valor possível, é maximizado o numerador. Note que, se forem explorados todos os valores possíveis para o elemento fixado dentro do tempo limite, é encontrada a solução ótima. O modelo MILP, para $\mathcal{H}$ corredores, a seguir descreve essa formulação.

$$\mathcal{F}(\mathcal{H}) = \max \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times p_o \quad (22)$$

$$s.t. \quad \sum_{a \in \mathcal{A}} y_a = \mathcal{H}, \quad \forall a \in \mathcal{A} \quad (23)$$

$$\sum_{o \in \mathcal{O}} w_{i,o} \times p_o \leq \sum_{a \in \mathcal{A}} q_{i,a} \times y_a, \quad \forall i \in \mathcal{I} \quad (24)$$

$$LB \leq \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o} \times p_o \leq UB \quad (25)$$

$$y_a, p_o \in \{0, 1\}, \quad \forall o \in \mathcal{O}, a \in \mathcal{A} \quad (26)$$

A função objetivo (22) maximiza o número de itens coletados para a quantidade de corredores definida previamente. A restrição (23) garante que o número de corredores seja exatamente $\mathcal{H}$ e as restrições (24) e (25) garantem a viabilidade do tamanho da *wave*.

A estratégia de busca proposta está descrita no Algoritmo 1. As iterações são divididas entre duas rotinas paralelas: ASCENDENTE, que incrementa, a partir de $h = 1$, o número de corredores fixado a cada iteração e DESCENDENTE, que decrementa, a partir de $H = |\mathcal{A}|$. A cada iteração, cada rotina resolve o modelo (22) e são atualizadas as soluções incumbentes X_{asc}^* e X_{desc}^* de suas respectivas rotinas. O algoritmo termina quando todo o intervalo discreto $[1, |\mathcal{A}|]$ é explorado, ou quando o tempo limite é atingido. A melhor solução encontrada é $\max\{X_{asc}^*, X_{desc}^*\}$.

Algorithm 1 Algoritmo Iterativo Paralelo

<pre> 1: $h \leftarrow 1$ 2: $H \leftarrow \mathcal{A}$ 3: $X_{asc}^*, X_{desc}^* \leftarrow 0$ 4: 5: function ASCENDENTE() 6: repeat 7: $X \leftarrow \mathcal{F}(h)$ 8: if X é viável then 9: $X_{asc}^* \leftarrow \max\{X, X_{asc}^*\}$ 10: $h \leftarrow h + 1$ 11: until $h \geq H$ </pre>	<pre> function DESCENDENTE() repeat $X \leftarrow \mathcal{F}(H)$ if X é viável then $X_{desc}^* \leftarrow \max\{X, X_{desc}^*\}$ $H \leftarrow H - 1$ until $H < h$ </pre>
--	---

7. Otimizações

Esta seção apresenta uma série de otimizações propostas para simplificar ainda mais o modelo que será otimizado pelo *solver*.

7.1. Redução de Variáveis e Restrições

Seja $Q_i = \sum_{a \in \mathcal{A}} q_{i,a}$ a quantidade total do item i ao longo de todos os corredores. Caso haja um pedido $o \in \mathcal{O}$ tal que, para algum $i \in \mathcal{I}_o$, $w_{i,o} > Q_i$, então esse pedido claramente não pode ser montado, e sua variável de decisão p_o é removida do modelo. Embora uma solução inteira na qual $p_o = 1$ seja obviamente inviável, a remoção da variável evita que valores fracionários sejam atribuídos a ela nas relaxações lineares da árvore *Branch-and-Cut* (B&C) [Conforti et al., 2014].

Outra forma de remover variáveis é por meio do conceito de *pedidos unitários*. Um pedido o^1 é dito ser unitário se, e somente se, existe $i \in \mathcal{I}_{o^1}$ tal que $w_{i,o^1} = 1 \wedge \forall i', i \neq i', w_{i',o^1} = 0$. Isto é, pedidos que requisitam apenas uma unidade de exatamente um tipo de item. Seja $\mathcal{O}_i^1$ o conjunto de todos os pedidos unitários formados pelo item i e $\bar{D}_i = |\mathcal{O}_i^1|$, a demanda do item i para pedidos unitários. Caso $\bar{D}_i > Q_i$, então é possível remover arbitrariamente $\bar{D}_i - Q_i$ pedidos pertencentes a $\mathcal{O}_i^1$ e suas respectivas variáveis de decisão, uma vez que implicam em um acréscimo equivalente na função objetivo e não impactam outras restrições.

Por fim, seja $\Theta \subseteq \mathcal{O}$ o conjunto de todos os pedidos que foram removidos por qualquer uma das estratégias acima, e i um item tal que $\forall o, w_{i,o} > 0 \Rightarrow o \in \Theta$. Em outras palavras, todo pedido que necessita de i foi removido do modelo. Isso significa que todas as restrições associadas a i podem ser removidas também, pois o lado esquerdo (LHS) dessas restrições $\leq$ sempre valerá zero. Chamaremos de Γ o conjunto de todos os itens que foram removidos por essa estratégia.

7.2. Redução de Variáveis Booleanas

Para todo pedido unitário que não foi removido pela estratégia acima, é desconsiderada sua restrição de integralidade. Assim, o escopo das variáveis p_o se torna:

$$0 \leq p_{o^1} \leq 1 \quad \forall o^1 \in \mathcal{O}^1 \setminus \Theta \quad (27)$$

$$p_o \in \{0, 1\} \quad \forall o \in \mathcal{O} \setminus \{\Theta \cup \mathcal{O}^1\} \quad (28)$$

A remoção das restrições de integralidade, nesse caso, preserva a validade do modelo. Se, em uma solução, para todo pedido unitário o^1 , $p_{o^1} = 1$, então a solução é inteira. Se não, se existe um conjunto de variáveis $\mathcal{P}$ que assumiram valor fracionário, então o somatório $\sum_{x \in \mathcal{P}} x$ pode ser fracionário ou inteiro. Se o primeiro caso for válido, então o valor objetivo $\mathcal{X}$ é fracionário e a solução não é ótima, pois é possível obter uma solução $\lceil \mathcal{X} \rceil$, uma vez que o limite da quantidade de itens a ser coletada é sempre inteiro. Se for inteiro, então basta que um pós-processamento escolha arbitrariamente $\mathcal{X} - \sum_{x \in \mathcal{P}} x$ variáveis de $\mathcal{P}$ para valer um e o resto, zero.

7.3. Minimização de Coeficientes

Coeficientes grandes podem levar o *solver* MILP a ter maior dificuldade em resolver o modelo. Portanto é preferível que os coeficientes sejam os menores possíveis. Um exemplo conhecido em que ocorre esse problema é no Método do Grande M. Esta subseção apresenta algumas estratégias para reduzir os coeficientes das restrições, preservando a validade do modelo.

Primeiramente, para todo i e a tal que a oferta $q_{i,a} > D_i$, onde $D_i = \sum_{o \in \mathcal{O}} \sum_{i \in \mathcal{I}_o} w_{i,o}$ é a demanda do item i por toda a instância, $q_{i,a}$ é atualizado para D_i . Além disso, quando $D_i > q_{i,a}$ e apenas um corredor a contém i , a tarefa de redução de coeficientes pode ser vista como a resolução de um *Maximum Subset Sum* em que é escolhido o subconjunto dos coeficientes do LHS da restrição que retorna a maior soma menor ou igual a $q_{i,a}$. Isso pode ser feito a partir da seguinte formulação de Programação Dinâmica, onde c é o vetor de coeficientes do LHS.

$$\mathcal{M}(j, \mathcal{Q}) = \begin{cases} -\infty, & \text{se } \mathcal{Q} < 0 \\ 0, & \text{se } \mathcal{Q} = 0 \\ \max(\mathcal{M}(j+1, \mathcal{Q}), c_j + \mathcal{M}(j+1, \mathcal{Q} - c_j)), & \text{se } j \in [1, |c|] \end{cases} \quad (29)$$

$$q_{i,a} = \mathcal{M}(1, q_{i,a}) \quad (30)$$

Ainda, seja $\bar{\mathcal{I}}$ o conjunto de todos os itens $\bar{i}$ tais que, para todo a , $q_{i,a} > 0 \Rightarrow q_{\bar{i},a} > D_{\bar{i}}$. Isto é, todo a que oferta $\bar{i}$ é capaz de suprir toda a demanda do item na instância. Assim, substituímos as restrições (5) por

$$p_o \leq \sum_{a \in \mathcal{A}_i} y_a \quad \forall i \in \bar{\mathcal{I}} \setminus \Gamma, \forall o \in \mathcal{O}_i \quad (31)$$

$$\sum_{o \in \mathcal{O}} w_{i,o} \times p_o \leq \sum_{a \in \mathcal{A}} q_{i,a} \times y_a \quad \forall i \in \mathcal{I} \setminus \{\bar{\mathcal{I}} \cup \Gamma\} \quad (32)$$

onde $\mathcal{O}_i \subseteq \mathcal{O}$ é o conjunto de todos os pedidos que dependem do item i e $\mathcal{A}_i \subseteq \mathcal{A}$, o conjunto de todos os corredores que contém o item i .

7.4. Atualização de Bounds e Redução de Iterações

Seja $v_{inc} = \frac{\mathcal{T}}{h}$ uma solução incumbente para o SPO em um dado instante da execução do algoritmo, em que $\mathcal{T}$ é o total de itens coletados e h , o número de corredores. Claramente, qualquer solução com $h' > h$ corredores consegue coletar pelo menos $\mathcal{T}$ itens. Porém, o algoritmo segue incrementando o número de corredores a fim de encontrar uma solução com valor objetivo maior que v_{inc} , e não apenas mais itens. Portanto, assim que uma solução viável é encontrada, toda iteração posterior tem o *lower bound* para o tamanho da *wave* atualizado para $\lceil v_{inc} \times h' \rceil$. Essa estratégia força o modelo a selecionar uma solução que colete um número de itens $\mathcal{T}'$ grande o suficiente para que $v_{h'} = \frac{\mathcal{T}'}{h'}$ supere v_{inc} , reduzindo o número de soluções viáveis (possivelmente todo o espaço de soluções) na iteração. O *upper bound*, por sua vez, é atualizado com o número máximo de itens da última iteração executada pela rotina descendente, se houver solução viável.

Além disso, é possível reduzir o intervalo de busca de novas soluções para $[h + 1, \lfloor \frac{UB}{v_{inc}} \rfloor]$, pois, a partir da iteração $\lfloor \frac{UB}{v_{inc}} \rfloor + 1$, mesmo que seja possível coletar UB itens, $UB / (\lfloor \frac{UB}{v_{inc}} \rfloor + \ell) < v_{inc}$, $\forall \ell > 0$. Caso a rotina descendente não esteja no intervalo, seu processo de otimização é simplesmente abortado e ela é transportada para a iteração $\lfloor \frac{UB}{v_{inc}} \rfloor$. Em um dado instante de execução, caso a rotina ascendente ainda não tenha encontrado solução viável e a descendente encontre uma iteração inviável, então o processo é finalizado, pois é notório que a solução incumbente da rotina descendente é ótima e nenhuma iteração seguinte será sequer viável.

Por fim, é proposta uma estratégia de “aceleração” da rotina descendente. Foi observado que, em instâncias viáveis, é comum que o limite superior de itens (UB) seja coletado ao longo de todo um intervalo $[H', |\mathcal{A}|]$. Para isso, o decremento unitário pode ser substituído por um decremento em $\beta \geq 1$ unidades. Isso evita que a rotina descendente gaste recursos em iterações redundantes, pois, se na iteração H , o tamanho máximo da *wave* é UB e, em $H - \beta$, também, então claramente este é o tamanho máximo da *wave* em todo o intervalo $[H - \beta, H]$. Para evitar que seja descartada a iteração em que é encontrada a solução ótima, o valor de β é atualizado para um quando ou a rotina descendente encontra uma solução com menos de UB itens ou ela é transportada para o intervalo $[h + 1, \lfloor \frac{UB}{v_{inc}} \rfloor]$ pela rotina ascendente. No primeiro caso, a rotina descendente ainda é transportada para a iteração $H - 1$, para explorar as iterações indevidamente descartadas. Ainda, se *par-it* é finalizado por *time out* em uma iteração $H - \beta$, então o intervalo considerado explorado é $[h, H - 1]$.

8. Método Híbrido

Descritos completamente a reformulação linear para o SPO, o algoritmo iterativo paralelo e as otimizações propostas, é possível montar um *pipeline* de resolução baseado em, inicialmente, reduzir o espaço de busca com o *par-it* e finalizar a otimização pelo *ref-lin*. Caso *par-it* não seja capaz de resolver o problema no tempo limite, restando apenas o intervalo de corredores $[h, H]$, o número de *lower* e *upper bounds* são atualizados na restrição (12), de acordo com as soluções encontradas nas rotinas, e é adicionada a seguinte restrição em *ref-lin*:

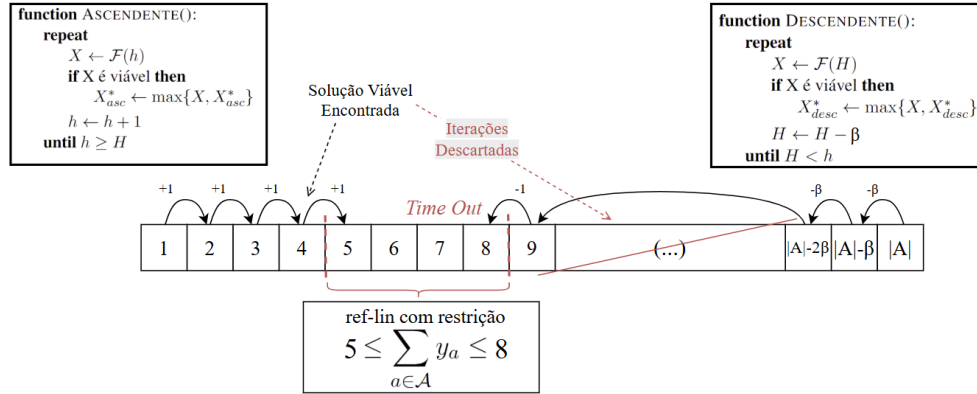


Figura 2: Pipeline de otimização proposto para o SPO.

$$h \leq \sum_{a \in \mathcal{A}} y_a \leq H, \quad \forall a \in \mathcal{A} \quad (33)$$

A Figura 2 representa graficamente este *pipeline*. O objetivo desta estratégia é combinar as vantagens dos dois métodos apresentados. Cada iteração de *par-it* resolve um subproblema significativamente menor que o SPO, ao fixar o número de corredores. No entanto, alguns subproblemas ainda podem ser intrinsecamente complexos, ou podem haver muitos corredores na instância original. Sendo assim, finalizar a resolução com *ref-lin* permite que o espaço de busca (reduzido) que restou seja otimizado por completo, encontrando soluções que estariam em iterações às quais *par-it* não teria tempo para alcançar.

9. Resultados Experimentais

Foram disponibilizadas pelo MERCADO LIVRE um total de 35 instâncias de teste³, e definido um limite de tempo de 10 minutos (600s) para cada uma. É garantido que todas as instâncias possuem solução viável.

Toda a solução foi implementada em Java 17, utilizando CPLEX 22.11 como *solver* MILP. Todos os testes foram executados em computador com um processador Intel® Core™ i7-12700H (2.30GHz) de 12ª geração, com 16G de memória RAM. Foi testada a resolução do SPO por ambos os métodos *par-it* e *ref-lin* e pela abordagem híbrida *par-it* + *ref-lin*. O parâmetro β foi considerado como $\lceil 1\% \times |\mathcal{A}| \rceil$. Para *par-it* + *ref-lin*, foram testados limites, de 30s em 30s, para a redução do espaço de busca. Os melhores resultados foram obtidos para 3 minutos e 30s (210s). Por questões de espaço, serão os únicos resultados apresentados para este método. Em *par-it*, cada rotina executou o *solver* com quatro *threads* disponíveis, e para *ref-lin*, foram disponibilizadas oito *threads*. Embora as abordagens sejam totalmente determinísticas, para considerar ligeiras variações em função do escalonamento de processos do sistema operacional, as abordagens que utilizam *par-it* foram executadas três vezes.

A Tabela 1 apresenta os resultados obtidos pelas três abordagens descritas. Cada linha apresenta os resultados de uma instância, cujo índice está na primeira coluna. Em seguida, as próximas três colunas apresentam o número de pedidos, itens e corredores, respectivamente. As colunas t (s) apresentam o tempo, em segundos, que cada algoritmo levou para resolver a instância. As colunas GAP apresentam a diferença percentual da solução encontrada pelo método em questão

³<https://github.com/mercadolivre/challenge-sbpo-2025>

para a solução ótima. Quando uma instância é marcada com †, significa que nenhum dos três métodos conseguiu garantir que a solução encontrada é ótima. Nesses casos, foi considerada a melhor solução encontrada dentre os três como referência. As instâncias marcadas com * são aquelas que foram resolvidas instantaneamente, nos referiremos a elas como “instâncias instantâneas”. As colunas h_{asc} e h_{desc} apresentam a próxima iteração que deixou de ser executada pelas rotinas ASCENDENTE e DESCENDENTE, respectivamente. Em par-it + ref-lin, este é o intervalo que será otimizado pelo modelo linearizado. Quando a solução ótima foi encontrada, o valor é omitido. Todos os dados das colunas par-it e par-it + ref-lin são valores médios das execuções.

	O	Z	A	par-it + ref-lin				par-it				ref-lin	
				t (s)	GAP	h_{asc}	h_{des}	t (s)	GAP	h_{asc}	h_{des}	t (s)	GAP
1 *	61	155	116	0	0	-	-	0	0	-	-	0	0
2 *	7	7	33	0	0	-	-	0	0	-	-	0	0
3 *	82	246	124	0	0	-	-	0	0	-	-	0	0
4 *	16	59	91	0	0	-	-	0	0	-	-	0	0
5	2625	6407	161	600	0	10	19	420	0	-	-	600	5
6	10341	7089	184	2	0	-	-	2	0	-	-	1	0
7	8320	5747	180	6	0	-	-	6	0	-	-	61	0
8	2185	5831	168	600	0	10	17	572	0	-	-	600	3
9 *	70	222	304	0	0	-	-	0	0	-	-	0	0
10 †	1602	3689	383	600	1	20	103	600	0	21	102	600	13
11 †	1029	2784	375	600	0	21	84	600	0	23	60	600	11
12 *	133	337	342	0	0	-	-	0	0	-	-	0	0
13	8375	7525	413	44	0	-	-	44	0	-	-	600	0
14	12402	10974	413	76	0	-	-	76	0	-	-	600	0
15	7367	6633	402	20	0	-	-	20	0	-	-	10	0
16 *	1108	1051	88	0	0	-	-	0	0	-	-	0	0
17 *	417	411	83	0	0	-	-	0	0	-	-	0	0
18	2682	2309	90	0	0	-	-	0	0	-	-	7	0
19 *	2257	2104	134	0	0	-	-	0	0	-	-	0	0
20 *	5	5	5	0	0	-	-	0	0	-	-	0	0
21	11321	7675	185	4	0	-	-	4	0	-	-	6	0
22	2162	5922	165	195	0	-	-	195	0	-	-	376	0
23	2070	5617	164	183	0	-	-	183	0	-	-	397	0
24 †	2448	6271	163	600	0	9	43	600	0	12	17	600	5
25	6468	4959	179	2	0	-	-	2	0	-	-	23	0
26 †	1857	4982	165	600	0	9	45	600	0	13	42	600	8
27 *	1841	1621	165	0	0	-	-	0	0	-	-	0	0
28	12334	10365	398	98	0	-	-	98	0	-	-	600	0
29	5581	4990	417	23	0	-	-	23	0	-	-	600	0
30	14952	13510	482	164	0	-	-	164	0	-	-	600	0
31 †	45112	37820	482	600	2	4	237	600	87	7	197	600	0
32 †	28263	24432	483	600	0	7	183	600	84	10	153	600	3
33 †	38214	32604	483	600	0	5	223	600	84	8	193	600	10
34 †	38202	32238	483	600	3	4	218	600	0	6	188	600	2
35	11541	9905	398	179	0	-	-	179	0	-	-	600	3

Tabela 1: Resultados experimentais para os três métodos estudados.

Dentre todas as 35 instâncias, 11 foram consideradas instantâneas e, em apenas oito, não foi garantida otimalidade. Das 27 instâncias resolvidas por pelo menos um método, o maior número de pedidos foi de 14952, de itens, 13510 e de corredores, 482, todos os valores se referindo à instância 30. Quando as rotinas de par-it não convergiram para uma iteração em comum, a abordagem explorou, em média, 75% do intervalo, e etapa iterativa de par-it + ref-lin, 73%. Porém par-it puro explorou completamente o intervalo de duas instâncias a mais.

A Figura 3 ilustra os tempos obtidos por cada método nas instâncias não-instantâneas e em que pelo menos uma das abordagens não foi finalizada por *time out*. O nível de dificuldade destas instâncias varia entre fácil e mediano, por não serem nem instantâneas, nem levarem todos os métodos a *time out*. Destas 16 instâncias, não houve empate das três abordagens em 14 e, em 13

destas, *par-it* obteve uma performance melhor que *ref-lin*, não sendo finalizada em *time out* nenhuma das vezes. Dentre todas estas 16 instâncias, na abordagem híbrida, a etapa *par-it* foi suficiente para resolvê-las. Nos outros dois casos, o método híbrido terminou em *time out*.

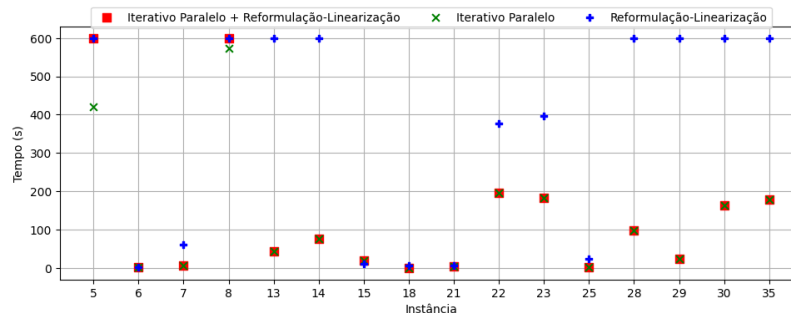


Figura 3: Tempo das instâncias não-instantâneas em que, em pelo menos um método, não houve *time out*.

O próximo gráfico, da Figura 4, apresenta os GAPs obtidos por cada método nas instâncias em que pelo menos uma das abordagens foi finalizada por *time out*. Primeiramente, vale observar que, nas instâncias em que o método híbrido não foi finalizado apenas executando a etapa *par-it*, a abordagem ainda assim retornou a solução ótima. Novamente, o gráfico conta com 16 instâncias, das quais houve empate dos três métodos em cinco. Destas 11 instâncias, *ref-lin* apresentou o pior GAP, dentre os três métodos, em sete, e *par-it*, em três. Nestas três instâncias, a abordagem apresentou um comportamento consideravelmente destoante dos resultados em outros testes. Esse resultado reflete a distância de soluções melhores em relação às extremidades do intervalo de busca, levando às rotinas a executarem muitas iterações inviáveis ou redundantes. Por outro lado, a abordagem *par-it* + *ref-lin* não só apresentou soluções significativamente melhores, como também prevaleceu dentre os três métodos em duas das três instâncias.

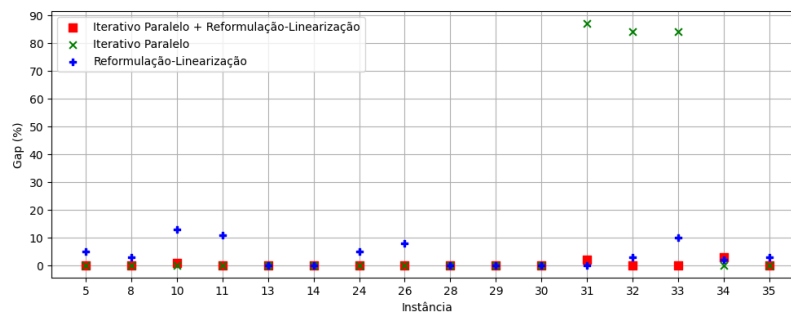


Figura 4: GAP das instâncias em que, em pelo menos um método, houve *time out*.

10. Conclusão e Trabalhos Futuros

Neste trabalho, foram desenvolvidas e comparadas três abordagens exatas para o SPO. A primeira, uma linearização da modelagem MILFP do problema. A segunda, um algoritmo iterativo e paralelo que, em cada iteração, otimiza um subproblema do SPO com exatamente $\mathcal{H}$ corredores. A terceira, um método híbrido que combina qualidades de ambas as abordagens anteriores.

Os resultados experimentais mostraram que todas as estratégias são capazes de retornar soluções ótimas em tempo hábil, sendo úteis tanto aplicação na indústria quanto para a avaliação de abordagens heurísticas. O algoritmo *par-it* foi o método que prevaleceu, em relação ao tempo, em 14 das 16 instâncias em que não houve empate. Por outro lado, nos casos em que as melhores iterações distam das extremidades do intervalo $[1, |\mathcal{A}|]$, o comportamento desta abordagem foi

significativamente pior que os dos outros métodos. Nestes casos, a reformulação linear conseguiu explorar mais amplamente as soluções, por não tratar múltiplos subproblemas redundantes. Por fim, o método híbrido foi capaz de combinar a redução do espaço de busca de *par-it* com a resolução ampla de *ref-lin*. Quando a própria etapa iterativa não foi suficiente para resolver o problema, a modelagem MILP encontrou soluções tão boas quanto, ou melhores, que *par-it* puro.

Em trabalhos futuros, é importante estudar o efeito do uso de abordagens heurísticas para o SPO no tempo de execução e nas soluções obtidas. Ainda, pretendemos estudar detalhadamente as características que impactam na dificuldade de resolução de uma instância e propor estratégias para contornar esses desafios.

11. Agradecimentos

Agradecemos ao MERCADO LIVRE e à Sociedade Brasileira de Pesquisa Operacional (SOBRAPO) pela proposta do desafio em parceria, e ao MERCADO LIVRE pela disponibilização das instâncias de teste.

Referências

- Charnes, A. e Cooper, W. W. (1962). Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3-4):181–186. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800090303>.
- Chen, F., Wang, H., Xie, Y., e Qi, C. (2016). An aco-based online routing method for multiple order pickers with congestion consideration in warehouse. *Journal of Intelligent Manufacturing*, 27(2):389–408. ISSN 1572-8145.
- Chen, M.-C. e Wu, H.-P. (2005). An association-based clustering approach to order batching considering customer demand patterns. *Omega*, 33(4):333–343.
- Conforti, M., Cornuéjols, G., e Zambelli, G. (2014). *Getting Started*, p. 1–44. Springer International Publishing, Cham. ISBN 978-3-319-11008-0. URL https://doi.org/10.1007/978-3-319-11008-0_1.
- Masae, M., Glock, C. H., e Grosse, E. H. (2020). Order picker routing in warehouses: A systematic literature review. *International Journal of Production Economics*, 224:107564. ISSN 0925-5273. URL <https://www.sciencedirect.com/science/article/pii/S0925527319304050>.
- Pardo, E. G., Gil-Borrás, S., Alonso-Ayuso, A., e Duarte, A. (2024). Order batching problems: Taxonomy and literature review. *European Journal of Operational Research*, 313(1):1–24. ISSN 0377-2217. URL <https://www.sciencedirect.com/science/article/pii/S0377221723001534>.
- Ratliff, H. D. e Rosenthal, A. S. (1983). Order-picking in a rectangular warehouse: a solvable case of the traveling salesman problem. *Operations research*, 31(3):507–521.
- Yang, P., Zhao, Z., e Guo, H. (2020). Order batch picking optimization under different storage scenarios for e-commerce warehouses. *Transportation Research Part E: Logistics and Transportation Review*, 136:101897. ISSN 1366-5545.
- Yue, D., Guillén-Gosálbez, G., e You, F. (2013). Global optimization of large-scale mixed-integer linear fractional programming problems: A reformulation-linearization method and process scheduling applications. *AIChE Journal*, 59.